

constexpr_if

Changes from previous version

- This paper revises N4461.
- Renamed static_if to constexpr_if.
- Added discussion about trying to solve the problem with existing facilities.

Abstract

I want to bring back parts of static if; namely bring it back in a form where it's

- restricted to block scopes.
- always going to establish a new scope.
- required that there exists values of the condition so that either condition branch is well-formed.

Why? Because it allows making static decisions without having to resort to multiple overloads. Having a static if allows for simple and local code, without having to know the intricacies of overload resolution, partial ordering and SFINAE.

Introduction

Richard Smith explained the following:

The "controversial" parts of N3329 are that:

- 1) it does not introduce a new scope, and
- 2) the non-selected branch is completely ignored (the tokens aren't even required to be parseable)

This makes it fundamentally incompatible with the template model used by at least two major implementations.

If, instead, it introduced a new scope (as proposed in this thread) and we had a requirement that it is possible to instantiate each arm of the static if (that is, the same requirement we have for other token sequences in templates), then I believe the over-my-dead-body objections from implementors would disappear.

So, the proposed constexpr_if (perhaps we could get rid of the space, it avoids splitting the keyword onto multiple lines. Not everyone uses clang-format. Yes, people who don't are foolish, but anyway) should have the characteristics Richard outlined.

Motivation

As the first example, I find it unwieldy to do pack unpacking with multiple overloads.

```
template <class T>
void f(T&& t)
{
    /* handle one T */
}

template <class T, class... Rest>
void f(T&& t, Rest&&... r)
{
    f(t);
    /* handle the tail */
    f(r...); // I think I have a bug here if I don't have a zero-param overload
}
```

It would be much simpler to be able to handle the unpacking in one function template, even though we're still writing recursive code.

```
template <class T, class... Rest>
void f(T&& t, Rest&&... r)
{
    /*
     * handle one T
     */
    constexpr_if (sizeof...(r)) {
        /*
         * handle the tail
         */
        f(r...); // I don't need a zero-param overload to do this
    }
}
```

Mutually exclusive constraints would also be arguably easier to grok. Instead of

```
template <class T, class... Args>
enable_if_t<is_constructible_v<T, Args...>, unique_ptr<T>>
make_unique(Args&&... args)
{
    return unique_ptr<T>(new T(forward<Args>(args)...));
}

template <class T, class... Args>
enable_if_t<!is_constructible_v<T, Args...>, unique_ptr<T>>
make_unique(Args&&... args)
{
    return unique_ptr<T>(new T{forward<Args>(args)...});
}
```

we could write

```
template <class T, class... Args>
unique_ptr<T>
make_unique(Args&&... args)
{
    constexpr_if (is_constructible_v<T, Args...>) {
        return unique_ptr<T>(new T(forward<Args>(args)...));
    } constexpr_else {
        return unique_ptr<T>(new T{forward<Args>(args)...});
    }
}
```

Even if the enable_ifs above are turned into constraints, I daresay the single-function solution is much simpler. A "damn sight nicer", if you ask me.

I expect there are many more good uses for such a facility than I can imagine. I have heard users hinting at wanting to write a function template that can take both signed and unsigned integral types, and write different code for the signed and unsigned cases, without having to worry about either branch emitting diagnostics even if never being taken - and those users do not think they want to write multiple overloads for integral types, since getting something like that right may end up being a heroic endeavor...

Can't we do this with existing language features?

John Spicer suggested in c++std-ext-17099 that polymorphic lambdas combined with a decision-making template would provide an adequate facility without a need to add new language features. The invocation of that decision-making template looks roughly like this:

```
template <int arg, typename ... Args> int do_something(Args... args) {
    return static_if<sizeof...(args)>::get(
        [](auto x, auto y) { return x+y; },
        [](auto x) { return *x; })(args...);
}
```

Now, in comparison, with the proposed language facility, we do

```
template <int arg, typename ... Args> int do_something(Args... args) {
    constexpr_if (sizeof...(args)) {
        return (args + ...);
    } constexpr_else {
        return *args...;
    }
}
```

Yes, I'm cheating - we nowadays have fold expressions. :) Without them, the equivalent code without would probably be written with a lambda, or by using a temporary tuple or array. I must point out some things here:

- I can return, break, continue and goto from within a constexpr_if block. I can not do that from within the lambda.
- While I am a big proponent of using lambdas to create new control facilities, I find the constexpr_if solution infinitely more readable.

Richard Smith explained the following:

Right, when a function template is instantiated, all of the declarations/statements/expressions within it are instantiated, and that includes pieces inside local classes, generic lambdas, and so on.

This instantiation of generic lambda bodies is in fact necessary for our language semantics -- computing the captures of a generic lambda within a function template specialization relies on us having already instantiated the complete closure type and its call operator template to the point that we know where the odr-uses are within the non-dependent full-expressions within the body.

In contrast, the intent of constexpr_if is that the branch not taken is not instantiated.

Yes, but don't Concepts provide a superior alternative?

Yes, for expressing the constraints of a function template, they do. No, for simplicity and locality of code, they don't. It's certainly easier to write (mutually exclusive and other) constraints with concepts, since it's possible to overload on concepts. The lack of locality remains, and the need to understand overload resolution, partial ordering and SFINAE remains. I posit that there are many simple cases where all that is still overly complex when a simple block-scope static condition would do much better. Chances are, of course, that combining Concepts with a constexpr_if can lead to expressive designs that are far superior to what either of these facilities can provide in isolation.

Implementability

Faisal Vali has implemented a prototype for clang, [here](#).

Teachability

Adding this facility will increase the overall complexity of the language, and since it's not identical or even very similar to the static if in D, it's not trivial to teach. I do have high hopes that it would be much simpler to teach for simple cases than using multiple overloads or using a metaprogramming facility with lambdas would be.

Wording

Very much TBD.

As a strawman description, a branch of a constexpr_if should be equivalent to an uninstantiated function template with the added twist that the declarations in the surrounding block scope (and in the condition) are visible to it. Is that correct?

Should the facility be restricted to appear only within function templates, member function templates, and member functions of class templates? Oh, and member templates of member classes of etc. etc.?